KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

MINI LIBRARY CATALOGUE

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
SHABIKSHA B	25BCS148	BSC.CS-2025
SARVESH S	25BCS146	BSC.CS-2025
SATHYAROOBA M	25BCS147	BSC.CS-2025
SHREE NISHA R	25BCS149	BSC.CS-2025
SHREEJA S	25BCS150	BSC.CS-2025

Academic Year: **2025 – 2026**

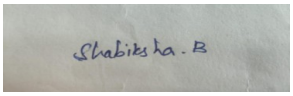
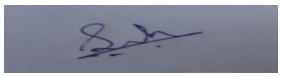
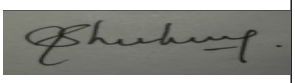
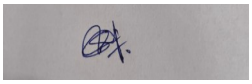
Semester: **I**

DECLARATION

We hereby declare that the project entitled "**MINI LIBRARY CATALOGUE"** submitted in partial fulfilment of the requirements for the course [**C PROGRAMMING]** is a **record of our original work** carried out under the guidance of **[MOSES JOHN PRAKASH]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
SHABIKSHA B	25BCS148	BSC.CS-2025	
SARVESH S	25BCS146	BSC.CS-2025	
SATHYAROOBA M	25BCS147	BSC.CS-2025	
SHREE NISHA R	25BCS149	BSC.CS-2025	
SHREEJA S	25BCS150	BSC.CS-2025	

Place: COIMBATORE

Date: 08-03-2026

CERTIFICATE

This is to certify that the project entitled **"MINI LIBRARY CATALOGUE"** is a **bona fide work** carried out by the following students of **[BSC,CS'B'-2025]**, [**COMPUTER SCIENCE]**, **KG College of Arts & Science**, in partial fulfilment of the requirements for the course **[C PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **[Month, Year]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	SHABIKSHA B	25BCS148
2.	SARVESH S	25BCS146
3.	SATHYAROOBA M	25BCS147
4.	SHREE NISHA R	25BCS149
5.	SHREEJA S	25BCS150

Guide:
(Technical Trainer, KGM)

MR.MOSES JOHN PRAKASH

Head of the Department
(Technical Head, KGM)

DR.BOOPALAN

Place: coimbatore

Date:08-03-2026

ABSTRACT

The Mini Library Catalogue System is a file-based application developed using the C programming language to manage book records efficiently. This system allows users to perform basic operations such as adding new books, displaying all available books, searching for a specific book, updating book details, and deleting records. It also provides additional features like counting the total number of books and listing books based on author name or publication year.

The program uses file handling techniques to store data permanently in a binary file, ensuring that the information is retained even after the program is closed. Each book record is stored using a structured format, making data management organized and efficient.

This project demonstrates the use of structures, file operations, and modular programming concepts in C. It is useful for understanding how real-world applications manage data and can be extended further into more advanced library management systems.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**6**
1.1	Objective	6
1.2	Overview	6
1.3	Features	6
2	**Program Analysis**	7
2.1	Problem Definition	7
2.2	Project Overview	7
2.3	Scopes and Limitations	7
3	**System Design**	**8**
3.1	Modules	8
4	**Methodology**	**8**
4.1	Problem Analysis	9
4.2	System Design	9
4.3	Program Development	9
4.4	Testing and Validation	9
4.5	Result and Documentation	10
4.6	Algorithm	10
4.7	Flowchart	11
5	**Source Code**	**13**
6	**Input and Output Screens**	**21**
7	**Discussion and Conclusion**	**22**
7.1	Discussion	22
7.2	Conclusion	22
8	**References**	**23**

1. INTRODUCTION

1.1 Objective

- To develop a system for storing and managing book details using file handling in C.
- To perform basic operations such as adding, searching, updating, and deleting book records.
- To demonstrate the use of structures and file operations for efficient data management.

1.2 Overview

This system allows users (cashiers or waiters) to:

1. The system is a simple library management program developed using the C programming language.

2. It stores book information such as ID, title, author, and publication year in a file for permanent storage.

3. The program provides various features like adding, viewing, searching, updating, and deleting book records.
4. It uses file handling and structured data to manage and organize records efficiently.

1.3 Features

1. Display all books
2. Search book by ID
3. Update book details
4. Delete book records
5. List books by author or year
6. Add new book records

2. PROGRAM ANALYSIS

2.1 Problem Definition

Managing book records manually is time-consuming and inefficient.

1. Searching and updating book details becomes difficult without a proper system.
2. There is a risk of data loss and errors in manual record keeping.
3. A simple and efficient system is needed to store and manage library data.

2.2 Project Overview

parameters	details
Language used	C programming
Compiler used	Turbo C
Concepts used	Structures, file handling
I/O type	Console input and output
Storage method	File-based storage

2.3 Scopes and Limitations

Scope:

- The system can be extended to include more features like user login, book issue/return, and fine calculation.
- It can be upgraded into a graphical user interface (GUI) or web-based application for better usability.

.

Limitations:

- No graphical user interface
- No user authentication

- Limited data handling capacity

3. SYSTEM DESIGN

3.1 Modules

1. Input Module
   Handles user input for adding and updating book details.
2. File Handling Module
   Manages storing, reading, and updating data in the file.
3. Search Module
   Searches for books based on ID.
4. Display Module
   Displays all book records in a formatted way.
5. Management Module
   Handles update and delete operations.
6. Filter Module
   Lists books based on author name or publication year.

Module Interaction Overview

Input Module → File Handling Module → Search/Filter Module → Display Module

4. METHODOLOGY

The system collects book details from the user through a menu-driven interface. It stores and manages the data using file handling techniques in C. Various operations like search, update, delete, and display are performed on the records efficiently.

4.1. Problem Analysis

Managing library records manually is a time-consuming and inefficient process. It becomes difficult to search and update book details without a proper system. Manual record keeping may lead to errors and possible data loss. There is no effective way to organize and retrieve information quickly. Therefore, a computerized system is required to manage records accurately and efficiently.

4.2. System Design

The system is designed as a menu-driven application using C programming. It uses structures to organize book data and file handling to store records permanently. The design is simple, modular, and easy to use.

Key Points

1. Menu-driven interface
2. Structured data storage
3. File-based system
4. Modular functions
5. Simple and user-friendly design

4.3. Program Development

The program is developed using the C programming language. It follows a modular approach with separate functions for each operation. Structures are used to organize book details efficiently. File handling is used to store and manage data permanently.

4.4. Testing and Validation

The program is tested with different inputs to ensure correct functionality. Each module is checked individually for accuracy and performance.

Valid and invalid inputs are used to verify error handling. File operations are tested to ensure proper data storage and retrieval. The output is verified to match the expected results. Overall, the system is validated for reliability and efficiency.

4.5. Result and Documentation

The system successfully manages book records efficiently.

All operations like add, search, update, and delete work correctly.

Data is stored and retrieved properly using file handling.

The output is clear and easy to understand.

Proper documentation is provided for understanding and future use.

4.6. Algorithm

An **algorithm** is a **step-by-step logical procedure** used to solve a specific problem or perform a computation.

In programming, it defines the exact sequence of actions the computer must follow to achieve the desired output from a given set of inputs.

Purpose of the Algorithm

The main purpose of using an algorithm in this project is to:

- Provides a clear step-by-step flow of the program
- Helps in organizing different operations logically
- Makes the program easy to understand and implement
- Ensures correct and efficient execution of tasks

Algorithm in the Project

Start the program

Display the menu options

Read user choice

If choice is 1, add a new book to the file

If choice is 2, display all books

If choice is 3, search for a book by ID

If choice is 4, delete a book by ID

If choice is 5, update book details

If choice is 6, count total books

If choice is 7, list books by author

If choice is 8, list books by year

If choice is 9, exit the program

Repeat the process until the user exits

Stop the program

Importance of Algorithm in the Project

- Helps to convert **real-world billing logic into programmable steps**.
- Ensures **systematic flow of control** and logical correctness.
- Acts as a **blueprint for coding** and debugging.
- Provides **clarity, modularity, and efficiency** in the program design.

4.7. Flowchart

A flowchart is a graphical representation of the step-by-step process of the Mini Library Catalogue System. It uses symbols and arrows to show the flow of operations such as displaying the menu, taking user input, performing actions like add, search, update, delete, and finally exiting the program. It helps in understanding the working of the program in a simple and visual manner.

Explanation of Flow

Start

The program begins execution.

Display Menu

All available options (Add, Display, Search, etc.) are shown to the user.

Enter Choice

The user selects an option from the menu.

Decision (Choice?)

The program checks which option the user selected.

Process **(Operation)**

Based on the choice:

- Add → new book is stored
- Display → all books shown
- Search → book is found
- Delete/Update → record is modified

Decision **(Continue?)**

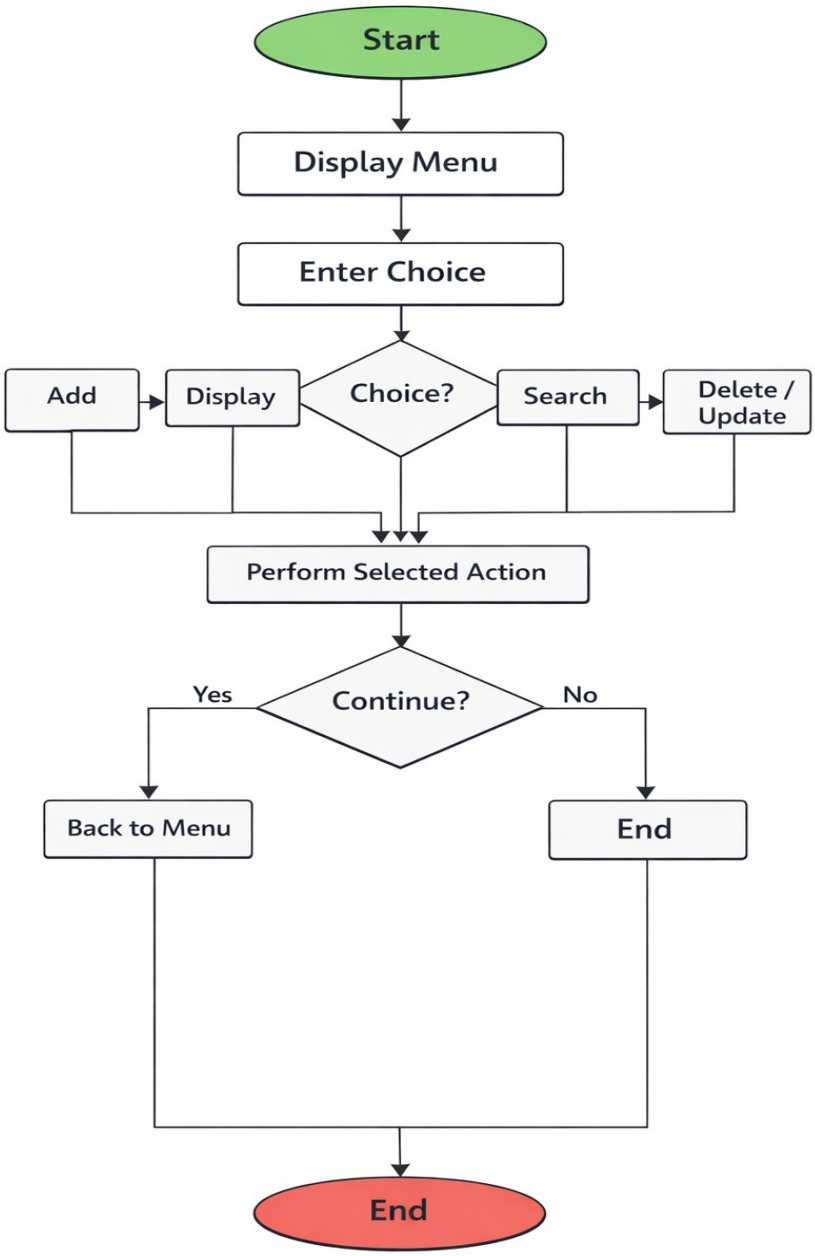
The system asks whether the user wants to perform another operation.

Yes

If yes, it goes back to the menu.

End

If no, the program stops.



5. SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FILENAME "library.dat"

typedef struct {
    int id;
```

```
    char title[100];
    char author[50];
    int year;
} Book;

// Function prototypes
void addBook();
void displayBooks();
void searchBook();
void deleteBook();
void updateBook();
void countBooks();
void listByAuthor();
void listByYear();
void menu();

int main() {
    int choice;
    while (1) {
        menu();
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        getchar(); // consume newline

        switch (choice) {
            case 1: addBook(); break;
            case 2: displayBooks(); break;
            case 3: searchBook(); break;
            case 4: deleteBook(); break;
            case 5: updateBook(); break;
            case 6: countBooks(); break;
            case 7: listByAuthor(); break;
            case 8: listByYear(); break;
            case 9: printf("\nExiting program... Goodbye!\n"); exit(0);
            default: printf("\nInvalid choice! Try again.\n");
        }
    }
    return 0;
}

void menu() {
    printf("\n========== MINI LIBRARY CATALOGUE ==========\n");
    printf("1. Add a New Book\n");
```

```
    printf("2. Display All Books\n");
    printf("3. Search Book by ID\n");
    printf("4. Delete Book by ID\n");
    printf("5. Update Book Details\n");
    printf("6. Count Total Books\n");
    printf("7. List Books by Author\n");
    printf("8. List Books Published After Year\n");
    printf("9. Exit\n");
    printf("==========================================\n");
}

void addBook() {
    FILE *fp = fopen(FILENAME, "ab");
    if (!fp) {
        printf("Error opening file!\n");
        return;
    }

    Book b;
    printf("Enter book ID: ");
    scanf("%d", &b.id);
    getchar();
    printf("Enter book title: ");
    fgets(b.title, 100, stdin);
    b.title[strcspn(b.title, "\n")] = '\0';
    printf("Enter author name: ");
    fgets(b.author, 50, stdin);
    b.author[strcspn(b.author, "\n")] = '\0';
    printf("Enter publication year: ");
    scanf("%d", &b.year);

    fwrite(&b, sizeof(Book), 1, fp);
    fclose(fp);

    printf("\nBook added successfully!\n");
}

void displayBooks() {
    FILE *fp = fopen(FILENAME, "rb");
    if (!fp) {
        printf("No books found!\n");
        return;
    }
```

```
    Book b;
    printf("\n%-5s %-25s %-20s %-5s\n", "ID", "Title", "Author", "Year");
    printf("-------------------------------------------------------\n");
    while (fread(&b, sizeof(Book), 1, fp))
        printf("%-5d %-25s %-20s %-5d\n", b.id, b.title, b.author, b.year);

    fclose(fp);
}

void searchBook() {
    FILE *fp = fopen(FILENAME, "rb");
    if (!fp) {
        printf("No books found!\n");
        return;
    }

    int id, found = 0;
    printf("Enter book ID to search: ");
    scanf("%d", &id);
    Book b;

    while (fread(&b, sizeof(Book), 1, fp)) {
        if (b.id == id) {
            printf("\nBook Found:\n");
            printf("ID: %d\nTitle: %s\nAuthor: %s\nYear: %d\n", b.id, b.title, b.author, b.year);
            found = 1;
            break;
        }
    }

    if (!found) printf("Book not found!\n");
    fclose(fp);
}

void deleteBook() {
    FILE *fp = fopen(FILENAME, "rb");
    FILE *temp = fopen("temp.dat", "wb");
    if (!fp || !temp) {
        printf("Error opening file!\n");
        return;
    }
```

```
    int id, found = 0;
    printf("Enter book ID to delete: ");
    scanf("%d", &id);
    Book b;

    while (fread(&b, sizeof(Book), 1, fp)) {
        if (b.id != id)
            fwrite(&b, sizeof(Book), 1, temp);
        else
            found = 1;
    }

    fclose(fp);
    fclose(temp);

    remove(FILENAME);
    rename("temp.dat", FILENAME);

    if (found)
        printf("Book deleted successfully!\n");
    else
        printf("Book not found!\n");
}

void updateBook() {
    FILE *fp = fopen(FILENAME, "r+b");
    if (!fp) {
        printf("Error opening file!\n");
        return;
    }

    int id, found = 0;
    printf("Enter book ID to update: ");
    scanf("%d", &id);
    getchar();

    Book b;
    while (fread(&b, sizeof(Book), 1, fp)) {
        if (b.id == id) {
            printf("Enter new title: ");
            fgets(b.title, 100, stdin);
            b.title[strcspn(b.title, "\n")] = '\0';
            printf("Enter new author: ");
```

```
            fgets(b.author, 50, stdin);
            b.author[strcspn(b.author, "\n")] = '\0';
            printf("Enter new year: ");
            scanf("%d", &b.year);

            fseek(fp, -sizeof(Book), SEEK_CUR);
            fwrite(&b, sizeof(Book), 1, fp);
            found = 1;
            break;
        }
    }

    fclose(fp);
    if (found)
        printf("Book updated successfully!\n");
    else
        printf("Book not found!\n");
}

void countBooks() {
    FILE *fp = fopen(FILENAME, "rb");
    if (!fp) {
        printf("No books found!\n");
        return;
    }

    Book b;
    int count = 0;
    while (fread(&b, sizeof(Book), 1, fp)) count++;
    fclose(fp);
    printf("Total number of books: %d\n", count);
}

void listByAuthor() {
    FILE *fp = fopen(FILENAME, "rb");
    if (!fp) {
        printf("No books found!\n");
        return;
    }

    char author[50];
    printf("Enter author name: ");
    getchar();
```

```
    fgets(author, 50, stdin);
    author[strcspn(author, "\n")] = '\0';

    Book b;
    int found = 0;
    printf("\nBooks by %s:\n", author);
    while (fread(&b, sizeof(Book), 1, fp)) {
        if (strcmp(b.author, author) == 0) {
            printf("ID: %d | Title: %s | Year: %d\n", b.id, b.title, b.year);
            found = 1;
        }
    }
    if (!found) printf("No books by this author.\n");
    fclose(fp);
}

void listByYear() {
    FILE *fp = fopen(FILENAME, "rb");
    if (!fp) {
        printf("No books found!\n");
        return;
    }

    int year;
    printf("Enter the year: ");
    scanf("%d", &year);
    Book b;
    int found = 0;
    printf("\nBooks published after %d:\n", year);
    while (fread(&b, sizeof(Book), 1, fp)) {
        if (b.year > year) {
            printf("ID: %d | Title: %s | Author: %s | Year: %d\n", b.id, b.title, b.author, b.year);
            found = 1;
        }
    }
    if (!found) printf("No books found.\n");
    fclose(fp);
}
```

SOURCE CODE EXPLANATION:

This program is a file-based library management system written in C. It helps to store and manage book details such as ID, title, author, and publication year using a file (library.dat).

- Structure Definition

A structure named Book is created to store book details:

id → unique book number

title → name of the book

author → author name

year → year of publication

- Main Function

The main() function runs an infinite loop to:

Display the menu

Take user input

Call the corresponding function using a switch-case

The program continues until the user selects the exit option.

- Menu Function

The menu() function displays different options like:

Add book

Display books

Search

Delete

Update

Count

List by author/year

- Add Book

The addBook() function:

Opens the file in append mode

Takes input from the user

Stores the book details in the file using fwrite()

- Display Books

The displayBooks() function:

Opens the file in read mode

Reads all records using fread()

Displays them in a formatted table

- Search Book

The searchBook() function:

Takes book ID as input

Searches the file

Displays the book details if found

- Delete Book

The deleteBook() function:

Copies all records except the selected book into a temporary file

Deletes the original file

Renames the temporary file

- Update Book

The updateBook() function:

Searches for a book by ID

Updates its details

Writes the new data back into the same position in the file

- Count Books

The countBooks() function:

Reads all records

Counts total number of books

- List by Author

The listByAuthor() function:

Takes author name as input

Displays all books written by that author

- List by Year

The listByYear() function:

Takes a year as input

Displays books published after that year

6. INPUT AND OUTPUT SCREENS

Input Screen 1:

```
======== MINI LIBRARY CATALOGUE =========
 1. Add a New Book
 2. Display All Books
 3. Search Book by ID
 4. Delete Book by ID
 5. Update Book Details
 6. Count Total Books
 7. List Books by Author
 8. List Books Published After Year
 9. Exit
==========================================
Enter your choice: 1
Enter book ID: 101
Enter book title: C Programming
Enter author name: Dennis Ritchie
Enter publication year: 1978
```

Input Screen 2:

```
Enter your choice: 3
Enter book ID to search: 101
--------------------------
Book Found:
  ID: 101
  Title: C Progrramming
  Author: Dennis Ritchie
  Year: 1978
```

Output Screen:

```
        Book added successfully!
-----------------------------------------
======== MINI LIBRARY CATALOGUE =========

Enter your choice: 2

-----------------------------------------------
 ID     Title                    Author     Year
-----------------------------------------------
101   C Programming    Dennis Ritchie   1978
-----------------------------------------------
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

• The program successfully performs all file-based operations.

• Data remains stored permanently even after program termination.

• The system is user-friendly and efficient for managing small libraries.

• Challenges included handling file updates and avoiding data duplication

7.2 Conclusion

• The project achieved its intended objectives.

• It provided a deep understanding of file handling and data management in C

8. REFERENCES

- Let UsC–Yashavant Kanetkar
- CProgramming–E. Balagurusamy
- OnlineTutorialsandDocumentatio